MAVIA — A Governed Multi-Agent System for Manufacturing Visual Inspection

Vidyesh Dapse · M.Tech CSE · September 2026 Repository: https://github.com/TejasDapse/MAVIA

Abstract

Automated visual inspection is mature at the detection layer, yet enterprises struggle to deploy it, because a defect classifier answers only one of the questions a quality process asks. MAVIA is a four-agent system that closes three specific gaps: it explains probable root cause by reasoning over retrieved precedent, it carries memory across production batches through a vector store of historical defect records, and it records every decision in a tamper-evident SHA-256 hash chain with a risk-routed human approval gate. Detection uses a from-scratch PatchCore implementation, reaching **0.9868 mean image AUROC** and **0.9767 pixel AUROC** across all fifteen MVTec AD categories at 35.5 ms per image — within 0.4 points of the published baseline. Retrieval reaches **0.455 precision@3 against a measured information ceiling of 0.449**, a ceiling established before any tuning by asking what an oracle could achieve on the same observable features. Withholding retrieved history changes the system's risk decision on **76%** of inspections and drops mean confidence from 0.436 to 0.100, demonstrating that the memory layer is load-bearing rather than decorative. The approval interrupt persists to disk and was verified resuming across three separate operating-system processes, with the audit chain intact across the whole sequence.

1. Introduction

Manual visual inspection is accurate to roughly 94% under ideal conditions, and degrades with fatigue, line speed, and task complexity. Automated inspection promises consistency, and modern vision systems detect sub-millimetre flaws beyond human perception. The economic case is well established: reductions in scrap and rework, improved first-pass yield, and lower recall exposure.

Yet adoption lags the technology. The obstacle is rarely detection accuracy. It is that a model emitting *pass* or *fail* does not fit into a quality management system, which asks three further questions of every finding:

1. **Why did this happen?** A defect is a symptom. Remediation targets a process.
2. **Has this happened before?** Recurrence changes both diagnosis and urgency.
3. **Can we prove what the system decided, and who agreed?** Regulated manufacturing requires traceable, attributable decisions.

MAVIA is built to answer all four questions — the detection plus these three — and to treat the last as a first-class engineering concern rather than logging bolted on at the end.

2. Problem Statement and Research Gap

Commercial systems (Landing AI, Overview AI, Cognex, Keyence) and the academic anomaly-detection literature both optimise the detection layer. Three gaps recur around it.

Gap	Consequence
No root-cause explanation	The line engineer receives a location, not a lead. Diagnosis remains manual.
No memory across batches	Each inspection is stateless. The system cannot recognise the fourth contamination event this week.
No governance layer	Decisions are not attributable or verifiable after the fact, making them unusable in an audited process.

Surveys of agentic AI adoption consistently identify security and monitoring gaps — not model capability — as the leading blockers to production deployment. MAVIA's contribution is not a better detector. It is a demonstration that the governance and reasoning layers around a strong detector can be built to a standard that is testable, measurable, and honest about its limits.

---

3. Related Work and Positioning

Unsupervised anomaly detection. MVTec AD (Bergmann et al., 2019, 2021) established the benchmark: defect-free training images only, pixel-precise masks for evaluation. PaDiM (Defard et al., 2020) models patch distributions; PatchCore (Roth et al., 2022) stores a coreset memory bank of patch embeddings and scores by nearest-neighbour distance, reaching ~99.1% image AUROC; EfficientAD (Batzner et al., 2024) trades a little accuracy for large latency gains.

Language-grounded vision. CLIP (Radford et al., 2021) enables zero-shot classification from text prompts; WinCLIP (Jeong et al., 2023) adapts this to anomaly detection for categories with no training data.

Retrieval-augmented generation. Lewis et al. (2020) established grounding generation in retrieved evidence. Sentence-BERT (Reimers & Gurevych, 2019) provides the embedding backbone used here.

Where MAVIA differs. The systems above solve detection or grounding in isolation. MAVIA composes them under a governance layer and — the part this report treats as its methodological contribution — **measures the ceiling of each component before optimising it**, which twice exposed evaluation bugs whose scores looked like success (§6.3).

---

4. System Design

```
[Product image]
      ↓
Agent 1 — Vision Inspector ....... PatchCore memory bank + CLIP zero-shot fallback
      ↓  VisionResult (verdict, score, regions)
Agent 2 — History Retriever ...... Qdrant hybrid search over defect history
      ↓  RetrievalResult (top-k comparable cases)
Agent 3 — Root Cause Analyst ..... Claude, structured output, verified citations
      ↓  RootCauseAnalysis (cause, action, risk_level, confidence)
      ↓
   risk ≥ HIGH  or  score ≥ threshold ?
      ├── yes → Human approval gate (LangGraph interrupt, persisted to SQLite)
      └── no  → Auto-proceed, fully logged
      ↓
Agent 4 — Report Writer .......... Markdown + HTML + PDF, audit chain embedded
      ↓
Audit trail ..................... Append-only SHA-256 hash chain
```

4.1 Design principles

1. **Typed contracts between agents.** Every agent consumes and produces a Pydantic model. No agent passes free-form dictionaries, which makes the graph statically checkable and the audit payloads self-describing.
2. **Every decision carries evidence.** No step emits a conclusion without a score, a latency, and a model identifier.
3. **Governance is not retrofitted.** The audit chain was built in Phase 1, before any model, so every later component was written against it.
4. **Degrade, never drop.** A missing API key, an unreachable vector store, or an unseen category produces a lower-confidence result with a recorded error. A production line cannot stop because a vendor is unavailable.

4.2 The central modelling decision

The original project brief specified YOLOv8. This was rejected, and the reasoning is the single most defensible technical decision in the project.

MVTec AD provides ~3,600 defect-free training images and ~1,700 test images. Defective samples appear **only** in the test split, and ground truth is supplied as segmentation masks, not bounding boxes. Training a supervised detector therefore requires splitting the test set to obtain defect examples, which contaminates evaluation — any reported mAP measures performance on data the model was tuned against.

It also mismatches the real problem. In a factory, defect-free product is abundant and each new failure mode is rare and initially unlabelled. A supervised detector finds only the classes it was shown. The operationally useful question — *is this unit unlike every good unit we have seen?* — is by construction an anomaly-detection question.

MAVIA therefore implements PatchCore, and reports image AUROC, pixel AUROC and PRO: the metrics the benchmark is actually scored with.

4.3 Implemented directly, not via a library

PatchCore is written from scratch (~600 lines) rather than imported from anomalib. Three reasons: anomalib carries PyTorch Lightning and a full training framework, while MAVIA computes no gradients anywhere; each step is separately testable, and is separately tested; and "I implemented the coreset selection and can explain why k-center beats random sampling here" is a different claim from "I used a library".

Load-bearing choices within it:

- **`layer2` + `layer3`, not `layer4`.** The deepest ImageNet features are specialised to classification and discard the local texture separating a scratch from a highlight.
- **3×3 neighbourhood aggregation**, so a patch descriptor carries local context and small misalignment does not register as anomalous.
- **Greedy k-center coreset** (Sener & Savarese, 2018) over random subsampling, with distances computed in a Johnson–Lindenstrauss projection for tractability.
- **Per-category threshold calibration**, since raw distances have no absolute scale — they depend on that category's own memory bank.

4.4 Governance mechanism

Each audit entry commits to its predecessor's digest:

```
payload_hash = SHA256(canonical_json(payload))
entry_hash   = SHA256(seq | inspection_id | timestamp | agent | action
                      | payload_hash | prev_hash)
```

The chain is **global rather than per-inspection**: a per-inspection chain would let an attacker delete an entire inspection undetected, whereas a global chain makes any removal visible as a sequence break. Auditing is structural — every graph node is wrapped before registration, so no node is responsible for logging itself and none can forget to.

---

5. Implementation

Component	Technology	Rationale
Detection	PatchCore on WideResNet-50-2, PyTorch	No gradients; new SKUs need only good samples
Zero-shot fallback	open_clip ViT-B-32	Covers a category before a memory bank exists
Vector memory	Qdrant, embedded on-disk	No daemon in development; one env var switches to a server
Embeddings	`all-MiniLM-L6-v2` (384-d)	Small enough for CPU at the edge
Orchestration	LangGraph + SQLite checkpointer	`interrupt()` and durable state are the governance requirement
LLM	Claude (`claude-opus-5`), structured output	`risk_level` drives control flow, so it must be an enum
Reporting	Hand-written templates → WeasyPrint	An audited document needs stable structure, not generated prose
Interface	Streamlit, with logic in a separate service layer	Streamlit re-executes on every interaction; UI is untestable
Packaging	Multi-stage Docker, non-root	Verified on `linux/aarch64`
Quality	ruff, strict mypy, pytest, GitHub Actions	134 unit tests in CI, plus integration

5.1 Notable implementation challenges

The dataset source was dead. The `mydrive.ch` URL that anomalib has used for years now returns 404, and search results still cite it. The download script was rewritten against an ungated Hugging Face mirror carrying the complete dataset (5,354 images, 1,258 masks — matching official counts exactly), with per-category count verification.

Sentence embeddings encode magnitude poorly. "covering 0.31%" and "covering 11.70%" differ by one token and embed close together, though one is a speck and the other a shattered part. This motivated hybrid retrieval (§6.2).

Checkpoint deserialisation. LangGraph's default serialiser accepts arbitrary types and warns that this will be blocked in future releases. MAVIA passes an explicit allowlist of its own state types, which keeps checkpoints loadable across upgrades and prevents a checkpoint database smuggling an arbitrary class past the loader.

---

6. Evaluation

All figures are reproducible from the repository; each section names the command.

6.1 Detection — MVTec AD, all 15 categories

`uv run python scripts/train_vision.py`

	This implementation	Roth et al. (2022)
Mean image AUROC	**0.9868**	0.991
Mean pixel AUROC	**0.9767**	0.981
Mean PRO	0.8046	—
Latency	35.5 ms/image (MPS)	—

Within ~0.4 points of the paper on both metrics, the expected margin for an implementation without the paper's test-time augmentation and multi-scale ensembling. Five categories reach a perfect 1.0000 image AUROC.

Weakest case, and why. `screw` scores 0.9332 against the paper's ~0.983. Screws are photographed at arbitrary rotations, so a memory-bank patch rarely aligns with the corresponding query patch, and the defects are small and low-contrast. Rotation augmentation of the memory bank is the standard remedy; it is listed in Future Work rather than quietly applied.

Why PRO is reported. Pixel AUROC is misleading on this task because defects occupy a tiny pixel fraction. On a controlled case with a 400 px defect and a 16 px defect, a model that finds the large one and misses the small one scores:

	PRO	Pixel AUROC
Both found	0.946	1.000
Small missed	0.493	0.978
Drop	**0.452**	**0.022**

Pixel AUROC falls two points while half the defects on the part were missed. PRO weights each connected region equally, which reflects the operational risk.

Why greedy k-center. With three clusters of 2,000 / 200 / **5** points and 1% selection, k-center retained the rare cluster in every trial; random sampling missed it in over half of fifty trials. A rare-but-normal appearance absent from the memory bank becomes a *false defect* at inference — so the algorithm is load-bearing, not an optimisation.

6.2 Retrieval

`scripts/retrieval_ceiling.py`, then `scripts/eval_retrieval.py`

The ceiling was measured first. The retriever sees only product category and defect geometry. Leave-one-out k-NN on the real per-mask features establishes what *any* retriever could achieve on that information:

	precision@3
Random within category	0.217
Oracle on real mask geometry (ceiling)	**0.449**

The ceiling is 0.449 rather than 1.0 because several defect types of the same product are genuinely indistinguishable by geometry: `bottle/broken_large` covers 11.7% ± 5.1 of the part, `bottle/contamination` 8.5% ± 5.5.

Configuration	precision@3	hit-rate@3	MRR
Random baseline	0.217	—	—
Dense only	0.3936	0.6795	0.5333
Hybrid α=0.7	**0.4548**	**0.6904**	**0.5521**
Geometry only (α=0.0)	0.4539	0.6822	0.5507
Oracle ceiling	*0.449*	*0.741*	—

Hybrid retrieval reaches the ceiling. Dense retrieval alone recovers 88% of it.

An honest reading of the α sweep. Pure geometry performs as well as the hybrid, meaning the dense embedding contributes little beyond separating product categories — unsurprising, since the observation text is a rendering of the geometry. This is a *structured* retrieval problem wearing semantic clothing. The vector store earns its place through category separation and extensibility, not semantic power, and the report says so rather than overclaiming.

6.3 Two measurement bugs the ceiling exposed

Both inflated the score while appearing healthy. They are recorded because the corrections matter more than the final number.

Version	precision@3	Defect
Gaussian-sampled corpus	0.498	Cases drawn from a per-mode fitted Gaussian, making synthetic history **more separable than real defects**
Bootstrapped, case-level split	0.539	Query and indexed cases could bootstrap the **same mask** — rewarding the retriever for finding its own duplicate
Bootstrapped, mask-level split	**0.394 → 0.455 hybrid**	Honest

The first scored 0.498 against a ceiling of 0.449 — *above the information-theoretic limit*, which is precisely how the bug was caught. Without computing the ceiling first, 0.539 would have looked like success. Both properties are now regression-tested.

6.4 Root-cause analysis

`uv run python scripts/eval_analysis.py --samples 25`

Metric	Result
Citation grounding rate	1.000
Action specificity	1.000
Risk agreement within one level	0.680
Risk under-called	**0.280**
Retrieval top-1 correct	0.440

The retrieval ablation. The same images analysed with and without history:

	With history	Without
Mean confidence	**0.436**	**0.100**
Risk level changed	—	**76% of cases**

Memory is load-bearing. Removing it changes the risk decision on three quarters of inspections and collapses confidence fourfold.

These numbers describe the deterministic fallback path, not Claude — no API key was configured for this run. That makes the ceiling explicit: fallback risk accuracy (0.480) is bounded by retrieval accuracy (0.440), because the fallback copies the nearest retrieved case's severity. The LLM path is expected to exceed this since it can weigh several cases against the detection evidence, but that remains a hypothesis until measured and is not claimed as a result. The 28% under-call rate is the figure that would matter on a real line, and it is reported rather than buried.

Guardrails, each tested with a stub client so they are verified deterministically: structured output into a Pydantic model; every cited `case_id` checked against what was actually retrieved, with invented citations stripped *and* the analysis confidence capped at 0.5; and degradation to the deterministic path on any API failure.

6.5 Orchestration and governance

Property	Result
Approval survives process restart	**Verified across 3 OS processes**
Escalation policy	Risk ≥ HIGH **or** score ≥ threshold (disagreement escalates)
Unrecognised resume payloads	10 shapes tested — all **fail closed** to rejection
Audit events per inspection	9–10, chain intact
Chain across host + container	**45 entries verified**
Tamper detection	Payload edits and deletions both caught

The durability result is the one that separates an agent from a script: `mavia inspect` suspended an inspection, `mavia pending` listed it from a second process, and `mavia approve` resumed it from a third — total latency 54 seconds, spanning a human decision, with the originating process long exited.

6.6 Packaging

3.28 GB image, built clean on `linux/aarch64` without Rosetta. Full inspection and PDF generation verified inside the container. The audit chain verifies across entries written by both the host CLI and the container, since the log is a mounted volume — the governance property holds across a runtime boundary.

---

7. Business Impact

Published industry figures place scrap and rework reductions from automated inspection at 18–30%, with corresponding first-pass-yield improvement and reduced recall exposure. MAVIA's specific contributions to that case:

- **Detection at 35.5 ms/image** on a laptop GPU (~28 units/second) without batching. The nearest-neighbour search is not the bottleneck; a 1% coreset holds only 470–3,065 entries per category.
- **Root-cause suggestions grounded in precedent** shorten the diagnosis loop, which is where engineering time is actually spent — detection is fast, attribution is slow.
- **Risk-routed escalation** concentrates scarce human attention on the inspections that warrant it, rather than all or none.
- **A tamper-evident record** is the precondition for using any of this in an audited quality process. Without it, the other three have no route to production.

These are the mechanisms by which value would be realised. This work does not measure them on a real line, and does not claim to.

---

8. Limitations and Future Work

Stated plainly, because a report that reads as uniformly successful is not credible.

1. **The defect-history corpus is synthesised.** Morphology is real — measured from MVTec's masks — but no public dataset records *why* those samples failed, so root causes are drawn from standard failure modes for the relevant processes. Retrieval metrics demonstrate that the mechanism works; they are not a claim about field accuracy.
2. **The LLM path is unmeasured.** All analysis figures describe the deterministic fallback. Running `scripts/eval_analysis.py` with a key configured fills this in.
3. **Tamper-*evident*, not tamper-*proof*.** An attacker with write access can recompute the whole chain. Production hardening would anchor periodic head digests in externally controlled append-only storage — a timestamping authority, a WORM bucket, or a transparency log.
4. **MVTec AD is a benchmark, not a line.** Fixed lighting, fixed pose, no conveyor motion blur. Real deployment requires domain adaptation.
5. **`screw` and `pill` underperform** for understood reasons (rotation invariance, legitimate colour variation). Rotation augmentation of the memory bank is the known remedy.
6. **Root-cause attribution is a plausibility judgement** over retrieved precedent. It is decision *support*; the human gate exists precisely because it is not decision *authority*.

Beyond these: EfficientAD for an accuracy/latency Pareto comparison, WinCLIP for stronger zero-shot coverage, drift detection on the anomaly-score distribution to catch process shift before defects appear, and a measured LLM-vs-fallback comparison on risk calibration.

---

9. Conclusion

MAVIA closes the three gaps identified in §2 with measured evidence rather than assertion: root-cause explanation grounded in precedent with verified citations, cross-batch memory whose removal demonstrably changes 76% of risk decisions, and a tamper-evident audit trail that survives process and runtime boundaries.

The methodological point outlasts the specific numbers. Measuring each component's ceiling *before* optimising it is what turned an apparently strong retrieval result into a caught bug — twice. A score above the information-theoretic limit is not a triumph; it is a defect report. Building the governance layer first, in Phase 1, is what made auditing structural rather than remembered. Both choices cost time early and repaid it repeatedly.

References

1. Bergmann, P., Fauser, M., Sattlegger, D., Steger, C. *MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection.* CVPR, 2019.
2. Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D., Steger, C. *The MVTec Anomaly Detection Dataset: A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection.* IJCV, 2021.
3. Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., Gehler, P. *Towards Total Recall in Industrial Anomaly Detection.* CVPR, 2022.
4. Defard, T., Setkov, A., Loesch, A., Audigier, R. *PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization.* ICPR Workshops, 2021.
5. Batzner, K., Heckler, L., König, R. *EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies.* WACV, 2024.
6. Sener, O., Savarese, S. *Active Learning for Convolutional Neural Networks: A Core-Set Approach.* ICLR, 2018.
7. Johnson, W. B., Lindenstrauss, J. *Extensions of Lipschitz mappings into a Hilbert space.* Contemporary Mathematics, 1984.
8. Zagoruyko, S., Komodakis, N. *Wide Residual Networks.* BMVC, 2016.
9. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L. *ImageNet: A Large-Scale Hierarchical Image Database.* CVPR, 2009.
10. Radford, A., et al. *Learning Transferable Visual Models From Natural Language Supervision.* ICML, 2021.
11. Jeong, J., Zou, Y., Kim, T., Zhang, D., Ravichandran, A., Dabeer, O. *WinCLIP: Zero-/Few-Shot Anomaly Classification and Segmentation.* CVPR, 2023.
12. Lewis, P., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS, 2020.
13. Reimers, N., Gurevych, I. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP, 2019.
14. Haber, S., Stornetta, W. S. *How to Time-Stamp a Digital Document.* Journal of Cryptology, 1991.
15. Merkle, R. C. *A Digital Signature Based on a Conventional Encryption Function.* CRYPTO, 1987.
16. LangGraph Documentation — Human-in-the-Loop and Persistence. LangChain, 2026.

17. Anthropic. *Claude API Documentation — Structured Outputs and Extended Thinking.* 2026.
18. Qdrant Documentation — Filtering and Hybrid Search. 2026.

Full source, evaluation scripts, and reproduction commands: https://github.com/TejasDapse/MAVIA